P1.2.- FUNDAMENTOS PRÁCTICOS

Librería GAlib de algoritmos genéticos: elementos básicos

Construcción de un algoritmo genético en GAlib para el problema de las n-reinas

Uso de la librería GAlib en Code::Blocks

Implementación de un algoritmo genético para el problema de las n-reinas

- ¿Cómo se comporta internamente el algoritmo genético?

Terminando de construir el sistema basado en el algoritmo genético para resolver el problema de las n-reinas

- Ajuste de parámetros: Estudio de casos – valores de parámetros
- Manual de asignación

Sobre la compilación con GAlib

Búsqueda mediante Algoritmos Genéticos

Recordemos, un esquema de un Algoritmo Genético (AG):




```
función Algoritmo Genético
  t = 0
  inicializar P(t)
  evaluar P(t)
  mientras (no condición de parada) hacer
        t = t + 1
        seleccionar P(t) desde P(t-1)
        cruzar P(t)
        mutación P(t)
        evaluar P(t)
```

Diseñaremos, construiremos y ajustaremos un AG mediante la librería GAlib:

- Es una librería de objetos C++,
- e incluye distintas representaciones y operadores genéticos.
- Material complementario:

https://www.um.es/web/innovacion/plataformas/ocw/listado-de-cursos/sistemas-inteligentes

Librería GAlib de algoritmos genéticos: elementos básicos

Los elementos básicos para resolver un problema usando la librería GAlib, y siguiendo el esquema de un AG, son:

1. Tipo de AG
   - En GAlib se usa la clase base **`GAGeneticAlgorithm`** (pag. 22)
   - Usaremos una de las clases derivadas de **`GAGeneticAlgorithm`** ("**`clase-tipoGA`**") (pag. 27, 29,31,32)
2. Representación de una solución del problema (individuo o cromosoma) usando un objeto
   - En GAlib se usa la clase base **`GAGenome`** (pag. 36)
   - Usaremos una de sus clases derivadas ("**`clase-GAGenome`**") (pag. 39,41,42,44,45, 47,48,50,52,54,57,58)
3. Inicializar una solución → "**`clase-GAGenome`**“ :: "**`método-de-inicialización`**"
4. Evaluar → lo define el usuario en función del problema
5. Selección
   - En GAlib se usa la clase base **`GASelectionScheme`** (pag. 77)
   - Usaremos una de las clases derivadas de **`GASelectionScheme`**(“**`clase-selección`**”)
6. Cruce → "**`clase-GAGenome`**" :: "**`método-de-cruce`**"
7. Mutación → "**`clase-GAGenome`**" :: "**`método-de-mutación`**"
8. Parada → "**`clase-tipoGA`**" :: "**`método-de-terminación`**"
9. Evolucionar la población → método **`evolve`** de **`GAGeneticAlgorithm`**

Librería GAlib de algoritmos genéticos: elementos básicos

Básicamente, los elementos dependen de dos clases: GAGenome, GAGeneticAlgorithm. Para definir los elementos de un AG, GAlib utiliza:

Individuo

- La clase **`GAGenome:`** se usa una de sus clases derivadas (**`"clase-GAGenome"`**)

  Inicialización
  - "**`clase-GAGenome`**" :: "**`método-de-inicialización`**"

  Operadores
  - Cruce → "**`clase-GAGenome`**" :: "**`método-de-cruce`**"
  - Mutación → "**`clase-GAGenome`**" :: "**`método-de-mutación`**"

Algoritmo Genético

- La clase **`GAGeneticAlgorithm`**
- Usaremos una de las clases derivadas de **`GAGeneticAlgorithm`** (**`"clase-tipoGA"`**)

  Selección
  - La clase base **`GASelectionScheme:`** se usa una de sus clases derivadas (**`"clase-selección"`**)

  Parada
  - "**`clase-tipoGA`**" :: "**`método-de-terminación`**"

  Evolucionar la población
  - método **`evolve`**

Librería GAlib de algoritmos genéticos: elementos básicos

INDIVIDUO

Seleccionada la "**`clase-GAGenome`**", se crea el individuo usando el constructor de la clase y se especifica la inicialización y operadores (cruce, mutación) mediante una de las opciones:

1. Dejando los operadores por defecto que asigna la librería a "**`clase-GAGenome`**". En este caso el usuario no debe hacer nada.
2. Usando otros operadores definidos en la librería para esa clase. En este caso sobre el individuo creado invocamos los métodos siguientes:
   - initializer("**`clase-GAGenome`**" :: "**`método-de-inicialización`**");
   - crossover("**`clase-GAGenome`**" :: "**`método-de-cruce`**");
   - mutator("**`clase-GAGenome`**" :: "**`método-de-mutación`**");
3. Implementando otros operadores. En este caso sobre el individuo invocamos los mismos métodos especificando como operadores métodos implementados por el usuario:
   - initializer(mi_metodo_inicio);
   - crossover(mi_metodo_cruce);
   - mutator(mi_metodo_mutacion);

   Las signaturas de estos métodos implementados deben ser:

   void mi_metodo_inicio(GAGenome &); parámetro: genoma a inicializar

   int mi_metodo_mutacion(GAGenome &, float); parámetros: genoma y probabilidad de mutación
   devuelve: número de mutaciones

   int funcion_mi_cruce(const GAGenome &, const GAGenome &, GAGenome*, GAGenome*);
   parámetros: dos genomas (entrada) y dos genomas (salida)
   devuelve: número de genomas (individuos) generados.

Librería GAlib de algoritmos genéticos: elementos básicos

INDIVIDUO

- ***Métodos***

Para acceder a los genes de un genoma podemos usar los métodos siguientes:

- `T & gene(unsigned int x=0)`

  Devuelve el valor del gen en la posición x

- `int length() const`

  Devuelve la longitud del genoma

Librería GAlib de algoritmos genéticos: elementos básicos

INDIVIDUO

- ***Ejemplo***

(y ésta derivada de **`GAGenome`**)

Clase **`GA1DArrayAlleleGenome`** derivada de **`GA1DArrayGenome`**

Constructor de **`GA1DArrayAlleleGenome`**:

```
GA1DArrayAlleleGenome<T>(unsigned int length, const GAAlleleSet<T> &
alleleset, GAGenome::Evaluator objective=NULL, void * userData=NULL)
```

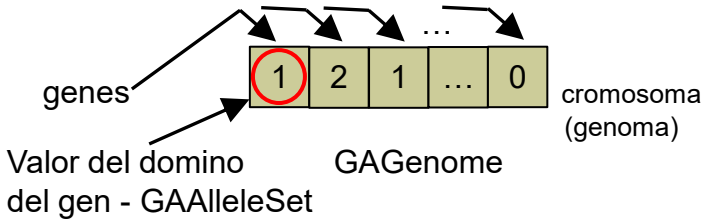
- Este constructor define un individuo o cromosoma (genoma) formado por un vector de objetos de tipo <T>, donde el valor de cada elemento (gen) está definido por un conjunto de alelos (dominio o conjunto de posibles valores para los genes).
- Como parámetros se indican:
  a) La longitud del cromosoma (número de genes): **`length`**
  b) El conjunto de alelos que definen el posible valor de cada gen (conjunto dominio): **`const GAAlleleSet<T> & alleleset.`** El dominio de los genes (conjunto de alelos) se define como un objeto de la clase **`GAAlleleSet<T>`** donde **`T`** indica el tipo de datos que representa los alelos.
  c) La función objetivo que define la función de idoneidad o fitness de cada cromosoma, y que tiene que ser definida por el usuario: **`GAGenome::Evaluator objective=NULL`**. Su signatura es **`float Objective(GAGenome &);`**

Librería GAlib de algoritmos genéticos: elementos básicos

d) Una estructura de datos con información del problema, si es necesaria: **`void * userData=NULL.`** Esa estructura puede ser recuperada desde los distintos operadores de la forma que se indica a continuación:

```
struct datos * Data=(struct datos *) genome.userData();
```

Gráficamente, un cromosoma lo podemos representar como:



Librería GAlib de algoritmos genéticos: elementos básicos

INDIVIDUO

- ***Ejemplo***

Inicialización de la población

- Indicamos el método para inicializar los individuos como sigue:

```
GA1DArrayAlleleGenome<T>::metodo-de-inicializacion
```

Por ejemplo, asignamos al individuo la inicialización uniforme:

```
genome.initializer(GA1DArrayAlleleGenome<T>::UniformInitializer);
```

Operadores

- Indicamos los operadores a usar como sigue:

```
GA1DArrayAlleleGenome<T>::metodo-de-cruce
GA1DArrayAlleleGenome<T>::metodo-de-mutacion
```

Por ejemplo, asignamos cruce por un punto y mutación aleatoria:

```
genome.crossover(GA1DArrayAlleleGenome<T>::OnePointCrossover);
genome.mutator(GA1DArrayAlleleGenome<T>::FlipMutator);
```

Librería GAlib de algoritmos genéticos: elementos básicos

ALGORITMO GENÉTICO

Creamos un AG de la clase derivada GASimpleGA usando el constructor:

```
GASimpleGA(const GAGenome &)
```

Este constructor crea un AG clonando el genoma que se le pasa como parámetro. Cada generación produce una nueva población seleccionando individuos de la generación anterior para que se reproduzcan.

Para indicar si queremos que el mejor individuo de la población anterior pase a la actual si no hay en la actual ningún individuo mejor usamos el método:

```
GABoolean elitist(GABoolean flag)
```

`flag` puede tomar los valores **`gaTrue`** o **`gaFalse`**. Por defecto está a true y por lo tanto el AG se define con elitismo.

Librería GAlib de algoritmos genéticos: elementos básicos

ALGORITMO GENÉTICO

Especificamos el método de selección a usar en el AG invocando sobre el mismo el método selector:

```
selector(const GASelectionSqueme & select);
```

donde select determina el esquema de selección y se define de la siguiente forma: `"clase-seleccion" select;`

Especificamos el operador de terminación de una forma equivalente a los operadores de cruce y mutación, pero invocando sobre el objeto algoritmo genético el método terminator:

```
terminator(GAGeneticAlgorithm :: "metodo-de-terminacion");
```

si el método de terminación es de la librería ó

```
terminator(metodo_terminacion);
```

si el método lo implementa el usuario. En este caso, la signatura del método es:

```
GABoolean metodo_terminacion(GAGeneticAlgorithm &);
```

El parámetro de entrada es el AG y devuelve true cuando se cumple la condición de parada.

Librería GAlib de algoritmos genéticos: elementos básicos

ALGORITMO GENÉTICO

- ***métodos***

Los principales métodos que podemos usar sobre un objeto algoritmo genético son los siguientes:

- **`evolve(unsigned int seed=0)`**
  Inicia la evolución del algoritmo genético hasta que se cumpla el criterio de parada.
- **`populationSize(unsigned int)`**
  Define el tamaño de la población.
- **`nGenerations(unsigned int)`**
  Define el número de generaciones.
- **`pCrossover(float)`**
  Define la probabilidad de cruce.
- **`pMutation(float)`**
  Define la probabilidad de mutación

Librería GAlib de algoritmos genéticos: elementos básicos

Operadores disponibles en la librería

Operadores primarios: operadores de inicio

- **UniformInitializer**: Inicializa el genoma siguiendo una distribución uniforme
- …

Operadores primarios: operadores de mutación

- **FlipMutator**: cambia el valor de un gen a otro de sus posibles valores
- **SwapMutator**: intercambia dos genes del individuo
- …

Operadores primarios: operadores de cruce

- **OnePointCrossover**: cruce por un punto.
- **TwoPointCrossover**: cruce por dos puntos
- …

Operadores primarios: métodos de selección

- **GARouletteWheelSelector:** Se asigna una probabilidad de selección proporcional al valor del fitness del cromosoma. (Modelo clásico)
- **GATournamentSelector:** Escoge al individuo de mejor fitness de entre Nts individuos seleccionados aleatoriamente con reemplazamiento (Nts=2,3,…)
- …

Operadores primarios: operadores de terminación

- **TerminateUponGeneration:** Condición de parada en base al número de generaciones.
- …

DISEÑO y CONSTRUCCIÓN de un AG en GAlib para el problema de las n-reinas

1. Representar una solución del problema (individuo o cromosoma) usando un objeto:
   - Nosotros usaremos la clase **`GA1DArrayAlleleGenome<T>`** **(pag. 41)**
2. Seleccionamos el método de inicialización.
   - **`GA1DArrayAlleleGenome<T>`** :: **`UniformInitializer`**
3. Evaluar → lo define el usuario en función del problema
4. **`GAGeneticAlgorithm`** → **`GASimpleGA`** **(pag. 31)**
5. Selección

   `GASelectionScheme` → **`GARouletteWheelSelector`** **(pag. 77)**

   Por defecto
6. Cruce → método **`crossover:`** **`GA1DArrayAlleleGenome<T>`** :: **`OnePointCrossover`**
7. Mutación → método **`mutator:`** **`GA1DArrayAlleleGenome<T>`** :: **`FlipMutator`**
8. Parada → método **`terminator:`** **`GASimpleGA`** :: **`TerminateUponGeneration`**
9. Evolucionar esa población para obtener la mejor solución.

   método **`evolve`** de la clase **`GAGeneticAlgorithm`**

Construcción de un AG en GAlib para el problema de las n-reinas

INDIVIDUO O CROMOSOMA:

Para representar a un individuo en el problema de las *n* reinas vamos a utilizar un array de *n* posiciones:

- Cada posición corresponde a una columna del tablero
- El contenido de la posición indica la fila en la que se encuentra situada la reina que corresponde a dicha columna
- En esta representación cada reina se mueve solamente en su columna

El conjunto de alelos (dominio de cada gen) está formado por el conjunto de posibles filas donde situar una reina (de 0 a *n*-1) con una longitud del cromosoma de *n* (para representar las *n* columnas)

- Utilizamos el método **`add(int i)`** de la clase **`GAAlleleSet<int>`**:

  Para las 8 reinas

```
GAAlleleSet<int> alelos;
for(int i=0;i<8;i++) alelos.add(i);
```

Construcción de un AG en GAlib para el problema de las n-reinas

Por tanto,

- Declaramos el tamaño del problema, definiendo la variable **`nreinas`**:

```
int nreinas=8;
```

- Definimos el dominio para los genes, creando un objeto **`alelos`** de la clase **`GAAlleleSet<T>`** que contiene el conjunto enumerado de alelos:

```
GAAlleleSet<int> alelos;
for(int i=0;i<nreinas;i++) alelos.add(i);
```

- Construimos un genoma (individuo), usando el constructor:

```
GA1DArrayAlleleGenome<int> mG(nreinas,alelos,Objective,NULL);
```

- La función idoneidad (o fitness), **`Objective`**, tendrá que ser definida por el usuario, y la explicaremos después

Construcción de un AG en GAlib para el problema de las n-reinas

INICIALIZACIÓN DE LA POBLACIÓN:

- La inicialización se lleva a cabo siguiendo una distribución uniforme:

  `GA1DArrayAlleleGenome<int>` :: `UniformInitializer`

OPERADOR DE MUTACIÓN:

- La mutación se lleva a cabo cambiando el valor de un gen por otro del conjunto de alelos:

  `GA1DArrayAlleleGenome<int>` :: `FlipMutator`

OPERADOR DE CRUCE:

- Usamos cruce por un punto:

  `GA1DArrayAlleleGenome<int>` :: `OnePointCrossover`

SELECCIÓN:

- Usamos la selección por ruleta:

  clase `GARouletteWheelSelector`

OPERADOR DE TERMINACIÓN:

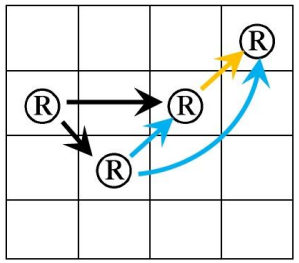
- La terminación se define en base al número de generaciones:

  `GAGeneticAlgorithm` :: `TerminateUponGeneration`

Construcción de un AG en GAlib para el problema de las n-reinas

FUNCIÓN FITNESS O IDONEIDAD

- Definimos la función fitness como el número de jaques que existen en el individuo
  - Un cromosoma será mejor cuanto menor sea el valor de esta función
  - El AG debe proporcionar la solución con menor valor (una solución con función fitness igual a 0 es una solución óptima)
- Los jaques se calcularán como se ilustra en la figura, contando una única vez los jaques mutuos entre dos reinas.
- La función fitness recibe un genoma como entrada y devuelve un valor para dicho genoma.



5 jaques

```
float Objective(GAGenome & g) {
  GA1DArrayAlleleGenome<int> & mG =(GA1DArrayAlleleGenome<int> &)g;
  float jaques=0;
   for(int i=0; i<mG.length(); i++)
      for(int j=i+1;j<mG.length();j++)
        if ((mG.gene(i)==mG.gene(j)||(abs(j-i)==abs(mG.gene(j)-mG.gene(i))))
            jaques++;
  return jaques;
}
```

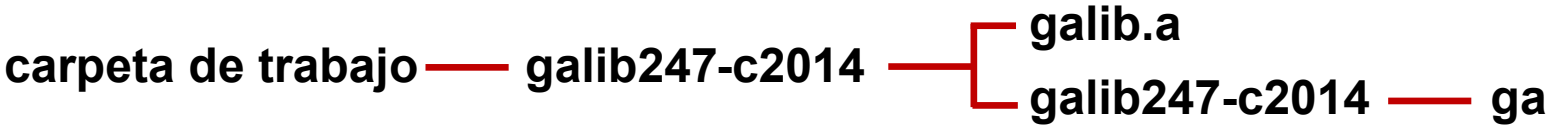
Uso de la librería GAlib en Code::Blocks

Para poder crear un proyecto en Code::Blocks que utilice la librería GAlib hay que seguir los siguientes pasos (ver también las transparencias 39 y 40)**:**

- Copiar la estructura de directorios, con su contenido, proporcionada en "**galib247-c2014.rar**"

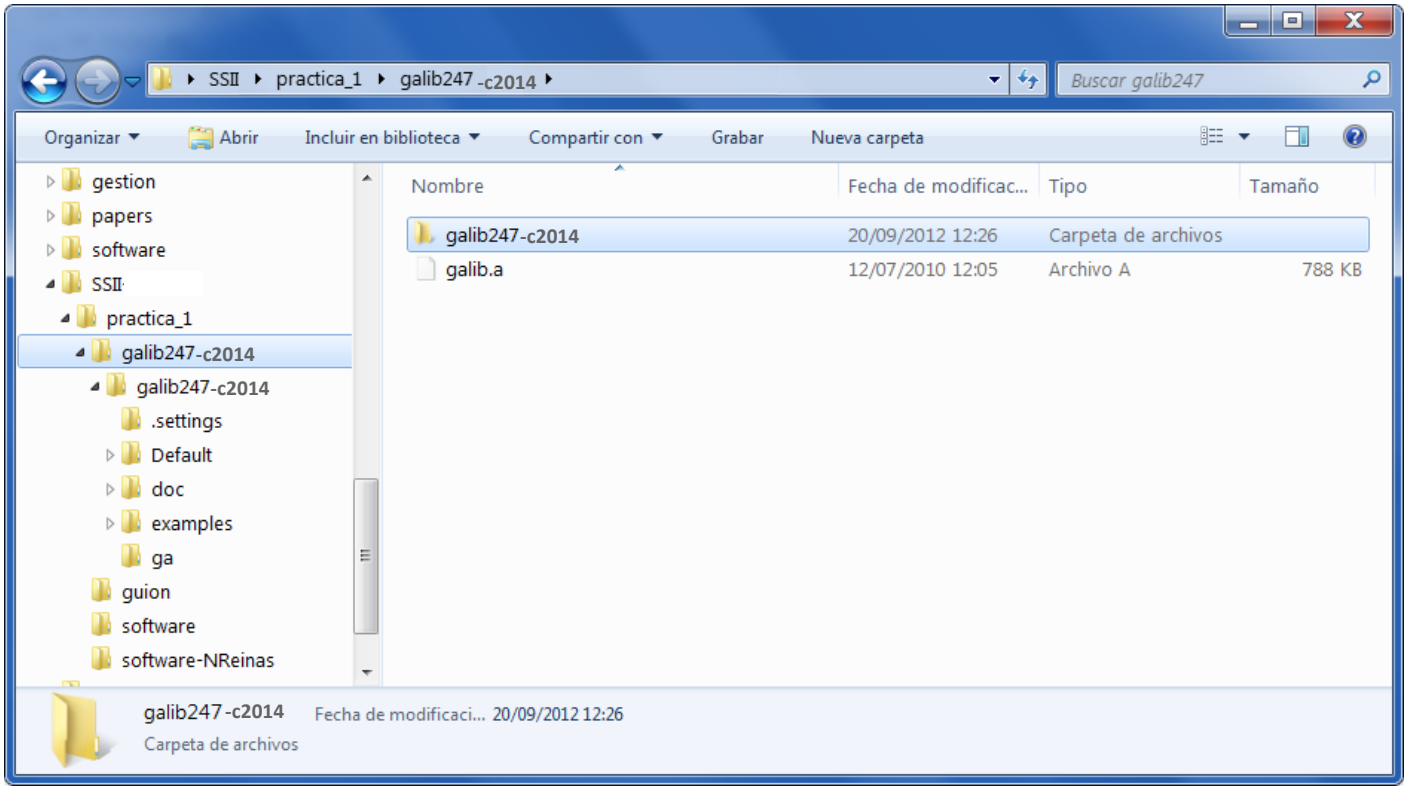
  La estructura:

  - El directorio "**`galib247-c2014`**" en la carpeta de trabajo
  - El directorio contendrá la librería estática **`galib.a`** y el subdirectorio "**`galib247-c2014`**"
  - Este nuevo subdirectorio contendrá el subdirectorio "**`ga`**" con los .c, .h, .o



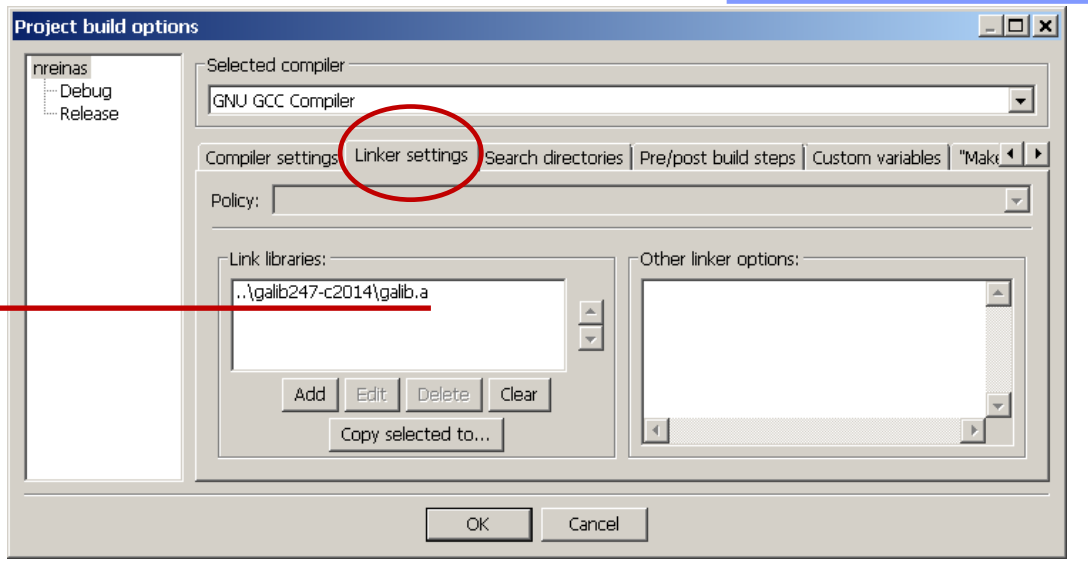
Uso de la librería GAlib en Code::Blocks

Por ejemplo, si llamamos `practica_1` al directorio de trabajo dentro de una carpeta creada para la asignatura `SSII`, la estructura de directorios sería la siguiente:

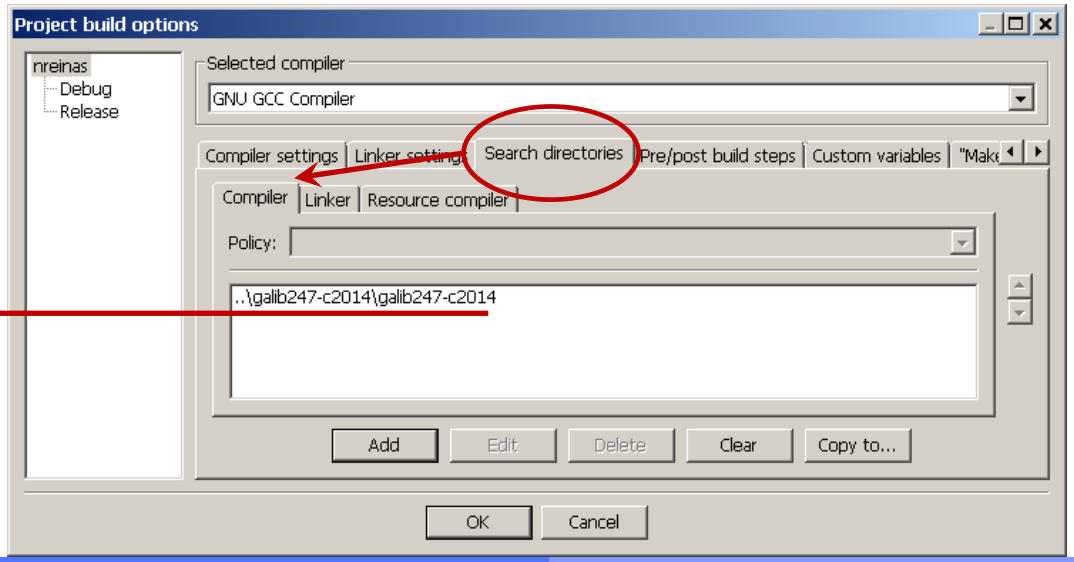


Uso de la librería GAlib en Code::Blocks

- Añadir la librería **`galib.a`** a las librerías estáticas en las opciones de proyecto



- Añadir el path para los includes



Implementación del AG para el problema de las n-reinas

El sistema implementado (basado en AG) tiene los parámetros (en orden):

número de reinas (nº entero)
tamaño de la población (nº entero)
número de generaciones (nº entero)
operador de selección (texto)
operador de cruce (texto)
operador de mutación (texto)
probabilidad de cruce (nº$\in$[0,1])
probabilidad de mutación (nº$\in$[0,1])

Veamos a continuación algunos detalles que hay que tener en cuenta en dicha implementación y el análisis de la salida del software.

Para los parámetros "operador de selección", "operador de cruce" y "operador de mutación" vamos a utilizar únicamente dos posibles valores:

- Op. selección: ROUL o TOUR (GARouletteWheel o GATournament)
- Op. cruce: ONEP o TWOP (OnePointCrossover o TwoPointCrossover)
- Op. mutación: FLIP o SWAP (FlipMutator o SwapMutator)

Métodos de búsqueda local y problemas de optimización

Implementación del AG para el problema de las n-reinas


```
/* ---------------------- PROBLEMA DE LAS N REINAS ---------------------- */
#include <ga/GASimpleGA.h>
#include <ga/GA1DArrayGenome.h>
#include <iostream>
#include <fstream>
using namespace std;
……..
float Objective(GAGenome &);
……..
int main(int argc, char **argv)
{  ……..
   int nreinas=atoi(argv[1]);
   ……..
   int popsize = atoi(argv[2]);
   int ngen = atoi(argv[3]);
   string sel = mayuscula(string(argv[4]));
   string cross = mayuscula(string(argv[5]));
   string muta = mayuscula(string(argv[6]));
   float pcross = atof(argv[7]);
   float pmuta= atof(argv[8]);
   ……..
```

Implementación del AG para el problema de las n-reinas


```
    // Creamos el genoma y definimos operadores de inicio, cruce y mutación
GA1DArrayAlleleGenome<int> mG(nreinas,alelos,Objective,NULL);
if (cross.compare("ONEP")==0) mG.crossover(GA1DArrayAlleleGenome<int>::OnePointCrossover);
else
    if (cross.compare("TWOP")==0) mG.crossover(GA1DArrayAlleleGenome<int>::TwoPointCrossover);
if (muta.compare("FLIP")==0) mG.mutator(GA1DArrayAlleleGenome<int>::FlipMutator);
else
    if (muta,compare("SWAP")==0) mG.mutator(GA1DArrayAlleleGenome<int>::SwapMutator);

    // Creamos el algoritmo genético
GASimpleGA ga(mG);
……..
ga.minimaxi(-1);
ga.populationSize(popsize);
ga.nGenerations(ngen);
ga.pCrossover(pcross);
ga.pMutation(pmuta);
if (sel.compare("ROUL")==0) {  GARouletteWheelSelector selector;   ga.selector(selector);   }
else
    if (sel.compare("TOUR")==0) {  GATournamentSelector selector;   ga.selector(selector);   }
ga.evolve(1);

    // Imprimimos el mejor individuo que encuentra el GA y su valor fitness
  ……..
}
……..
```

Siempre antes de crear el AG

Utilizamos en el parámetro de entrada el valor 1

Implementación del AG para el problema de las n-reinas


```
/* ---------------------- PROBLEMA DE LAS N REINAS ---------------------- */
#include <ga/GASimpleGA.h>
……..

int main(int argc, char **argv)
{
   ……..
   ga.nGenerations(ngen);
   ……..

   ga.evolve(1);
   ……..
}
……..
```

ESPECIFICANDO EL OPERADOR DE TERMINACIÓN

Por defecto

```
GABoolean Termina(GAGeneticAlgorithm & ga)
 {
   if (ga.statistics().generation()==ga.nGenerations()) return gaTrue;
   else return gaFalse;
 }
```

Implementación del AG para el problema de las n-reinas

ESPECIFICANDO EL OPERADOR DE TERMINACIÓN

```
/* ---------------------- PROBLEMA DE LAS N REINAS ---------------------- */
#include <ga/GASimpleGA.h>
……..
GABoolean Termina(GAGeneticAlgorithm &);

int main(int argc, char **argv)
{
   ……..
   ga.nGenerations(ngen);
   ……..
   ga.terminator(Termina);
   ga.evolve(1);
   ……..
}
……..
 // Funcion de terminacion

GABoolean Termina(GAGeneticAlgorithm & ga)
{
   if ( ga.statistics().minEver()==0 ) return gaTrue;
   else return gaFalse;
}
```

Implementación del AG para el problema de las n-reinas


```
/* ---------------------- PROBLEMA DE LAS N REINAS ---------------------- */
#include <ga/GASimpleGA.h>
……..
GABoolean Termina(GAGeneticAlgorithm &);

int main(int argc, char **argv)
{
   ……..
   ga.nGenerations(ngen);
   ……..
   ga.terminator(Termina);
   ga.evolve(1);
   ……..
}
……..
 // Funcion de terminacion

GABoolean Termina(GAGeneticAlgorithm & ga)
{
   if ( (ga.statistics().minEver()==0) || (ga.statistics().generation()==ga.nGenerations()) ) return gaTrue;
   else return gaFalse;
}
```

ESPECIFICANDO EL OPERADOR DE TERMINACIÓN

Implementación del AG para el problema de las n-reinas

Ejemplo de uso (salida):

Obsérvese que, en el código, se numeran las filas y columnas de 0 a 7.

¿mostramos el genoma?

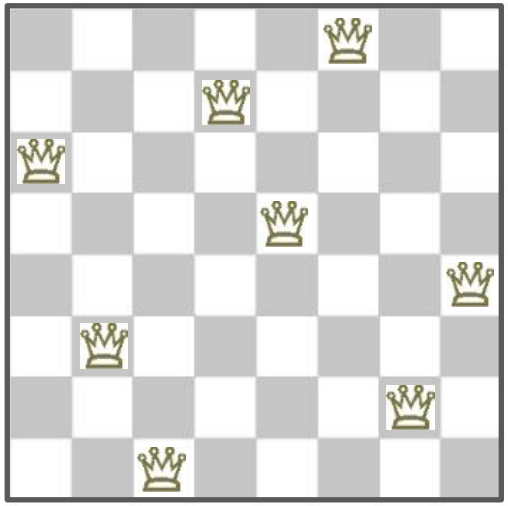
Problema de las 8 reinas

Parámetros: (100, 1000, ROUL, ONEP, FLIP, 0.9, 0.1)

El sistema encuentra la solución

(2 5 7 1 3 0 6 4)

con 0 jaques.



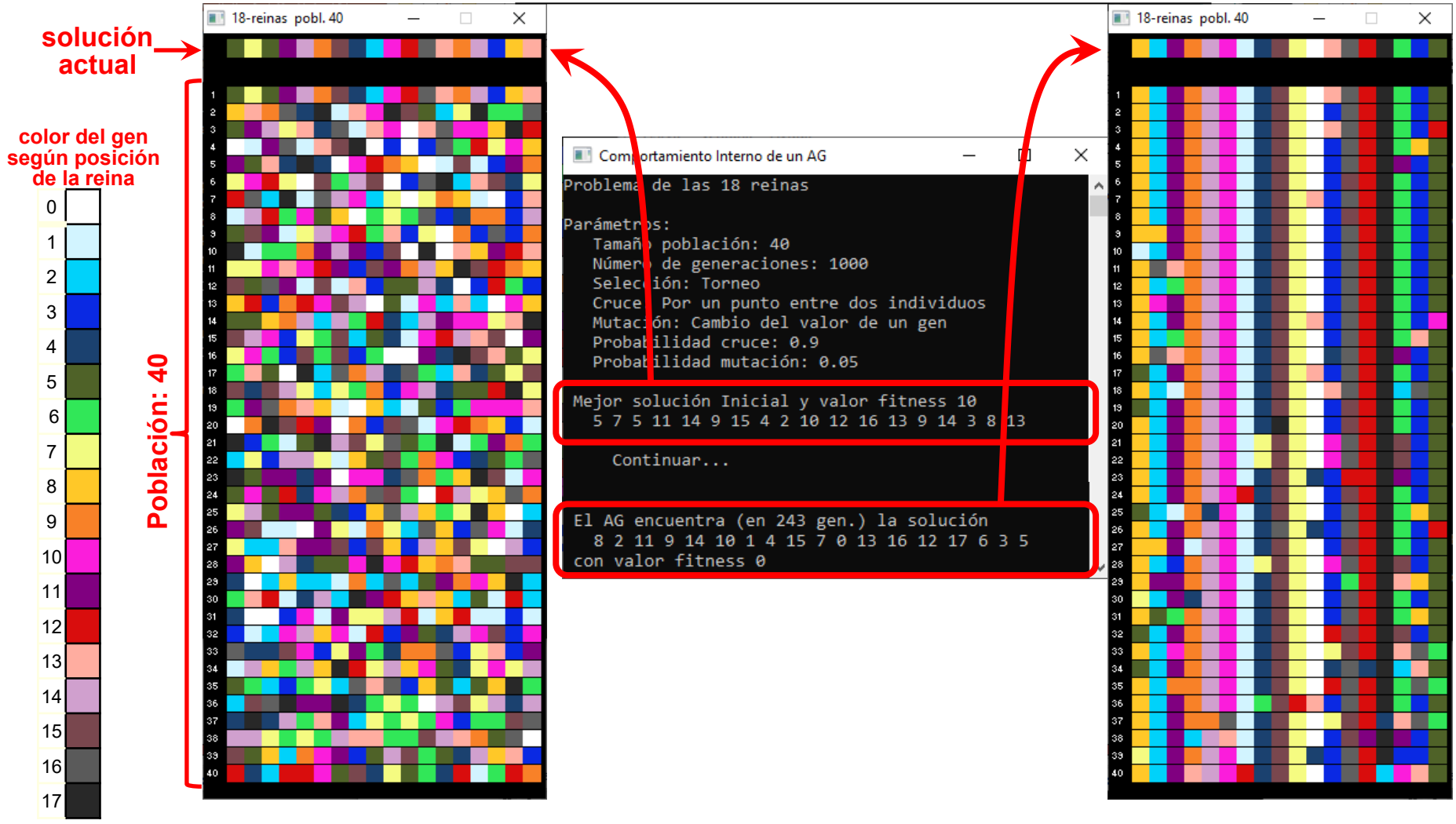
Es solución óptima, pues fitness es 0

2	5	7	1	3	0	6	4

Ojo, ¿es interpretable la solución para un usuario final? ¿Por qué?

Implementación del AG para el problema de las n-reinas

¿CÓMO SE COMPORTA INTERNAMENTE EL ALGORITMO GENÉTICO?



TERMINANDO de construir el sistema basado en AG (ajuste de parámetros)

Para terminar el sistema basado en el AG diseñado y construido debemos:

- Encontrar las mejores combinaciones de operadores genéticos-valores de parámetros que hagan que el AG tenga un comportamiento óptimo, "bueno" o "aceptable". Este será el objetivo del ajuste de parámetros.
- A partir de este ajuste, debemos construir un manual que le sirva al usuario final a decidir, para un problema dado, los valores concretos de parámetros y operadores que debe utilizar (Manual de Asignación)

Vamos como comenzar a realizar el ajuste. Para ello, vamos a realizar un análisis inicial para decidir distintos valores para los valores y detectar patrones en relación al comportamiento del AG en la resolución de las n-reinas.

AJUSTE DE PARÁMETROS: ESTUDIO DE CASOS – VALORES DE PARÁMETROS

Como ya hemos comentado, una vez diseñado/construido el AG para resolver el problema de las n-reinas, debemos realizar el ajuste de parámetros. A continuación describimos los pasos para este ajuste:

a) Parámetros del AG.

Parámetros con valores fijos: nº generaciones = 10000
operador de cruce = ONEP
operador de mutación = FLIP

Parámetros ajustables con valores:

→ tamaño de la población (TP)= {100,150}
→ método de selección = {ROUL, TOUR}
→ p_c = {0.85, 0.9}
→ p_m = {0.025, 0.05, 0.075, 0.1}

b) Conjunto de casos del problema: de 8 a 40 reinas.

c) Resolvemos todos los casos utilizando todas las combinaciones de valores.

Terminando de construir el sistema basado en un AG

1

Selector = ROUL

Nreinas	p_m	TP=100								TP=150							
		p_c=0.85				p_c=0.9				p_c=0.85				p_c=0.9			
		0.1	0.075	0.05	0.025	0.1	0.075	0.05	0.025	0.1	0.075	0.05	0.025	0.1	0.075	0.05	0.025
8		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11		0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0
12		0	0	0	0	1	0	0	0	0	0	1	0	1	1	0	0
13		0	0	0	1	1	0	1	0	1	1	0	0	1	1	0	0
14		1	1	0	0	0	0	0	0	1	1	0	1	1	1	0	0
15		0	0	0	0	1	1	1	0	1	1	1	0	1	1	0	0
16		1	1	0	0	0	0	1	0	1	1	0	0	0	0	1	0
17		1	1	0	0	1	1	0	0	1	1	0	0	2	2	0	1
18		1	1	1	0	1	1	0	1	1	1	0	0	1	1	0	0
19		1	1	0	0	1	1	1	0	1	1	1	0	2	2	1	0
20		1	1	0	0	2	2	0	0	1	1	0	0	2	2	1	1
21		3	2	2	0	3	1	1	0	2	2	2	0	2	2	2	1
22		2	1	1	0	3	1	1	0	3	2	2	1	2	1	1	0
23		4	0	0	0	3	1	1	1	3	2	2	1	4	2	2	1
24		3	2	2	0	3	2	2	1	3	2	2	1	2	1	1	1
25		2	2	2	0	5	2	2	1	5	3	3	0	3	3	3	1
26		5	3	3	0	5	1	1	1	4	3	3	1	5	3	3	1
27		4	4	2	0	4	4	2	1	3	3	3	2	3	3	3	2
28		4	4	2	0	4	4	2	1	5	5	3	1	4	4	3	1
29		2	2	2	0	6	6	3	1	4	4	1	2	5	5	2	1
30		6	6	3	0	4	4	2	2	6	6	3	1	3	3	3	1
31		7	5	2	1	7	5	3	1	7	6	3	0	8	6	4	2
32		6	4	2	2	8	8	3	1	7	5	2	1	9	5	3	1
33		6	7	3	1	8	5	4	2	8	4	4	1	8	5	4	2
34		7	6	2	1	8	6	4	3	9	4	4	2	8	7	4	3
35		6	8	4	1	8	5	4	1	9	5	4	1	8	6	5	1
36		8	6	4	3	9	5	4	2	9	6	3	1	10	5	4	1
37		8	7	4	1	9	8	5	2	10	6	6	1	10	7	3	2
38		8	6	4	4	10	7	4	3	7	6	4	4	8	9	3	1
39		9	7	5	2	11	7	7	1	8	7	7	3	9	7	5	3
40		10	6	4	2	10	7	8	3	10	8	6	2	8	7	5	1

Terminando de construir el sistema basado en un AG

2

Selector = TOUR

Nreinas	p_m	TP=100								TP=150							
		p_c=0.85				p_c=0.9				p_c=0.85				p_c=0.9			
		0.1	0.075	0.05	0.025	0.1	0.075	0.05	0.025	0.1	0.075	0.05	0.025	0.1	0.075	0.05	0.025
8		0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
9		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
11		0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
12		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
13		0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
14		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
16		0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
17		1	1	0	0	1	1	0	0	1	1	0	0	0	0	0	0
18		1	1	0	0	1	1	0	0	1	1	0	0	0	0	0	0
19		1	1	0	0	0	0	0	0	0	0	0	0	1	1	0	0
20		0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0
21		2	0	0	0	2	0	0	0	1	1	1	0	3	0	0	0
22		3	0	0	0	2	1	1	0	2	0	0	0	2	0	0	0
23		2	1	1	0	2	1	1	0	2	1	1	0	1	1	1	0
24		2	1	1	0	2	1	1	0	3	1	1	0	2	1	1	0
25		4	1	1	0	2	2	2	0	3	1	1	0	2	2	2	0
26		3	2	2	0	3	1	1	0	4	1	1	0	3	1	1	0
27		3	3	1	0	4	4	1	0	4	4	1	0	2	2	1	0
28		3	3	1	0	3	3	1	0	3	3	2	0	4	4	2	1
29		3	3	1	0	5	5	1	0	3	3	1	0	3	3	1	0
30		5	5	2	0	4	4	2	0	4	4	1	0	3	3	2	0
31		7	3	2	0	7	2	1	0	6	4	2	0	5	5	2	0
32		6	4	1	0	6	4	1	0	6	4	1	0	5	5	2	0
33		6	4	2	0	6	5	2	0	5	2	2	0	6	3	3	0
34		6	6	2	0	8	4	3	0	7	5	2	0	7	6	3	0
35		6	5	3	0	7	5	2	0	5	3	2	0	7	3	2	0
36		7	5	2	0	7	5	1	0	5	4	1	0	8	4	4	0
37		8	5	3	0	8	5	3	0	7	6	1	0	8	5	3	0
38		8	6	2	0	7	5	2	0	7	4	2	0	8	4	2	0
39		9	4	2	0	7	5	3	0	7	4	2	0	6	6	2	0
40		6	5	3	0	6	6	3	0	8	3	3	0	8	6	3	0

Terminando de construir el sistema basado en un AG

d) se analizan los resultandos obtenidos (Análisis de Resultados)

Las decisiones tienen que estar guiadas por los resultados y fundamentadas por los criterios de "Optimalidad“ y las complejidades en "Tiempo" y en "Espacio".

Con el comportamiento que observemos, debemos realizar una generalización del proceso de asignación de valores a parámetros y casos a resolver.

- Primero marcamos las combinaciones de valores que obtienen un "0" para todos los casos resueltos (buscando la optimalidad).
- En la Tabla 2 se observan más soluciones 0 que en la Tabla 1. Y además, para los casos que no obtienen "0" tienen un número de "jaques" sensiblemente menor que los de la Tabla 1. Por tanto, el operador de selección "TOUR" (**GATournament**) tiene un mejor comportamiento que el operador de selección "ROUL" (GARouletteWheel).
- Centrándonos ahora en los resultados de la Tabla 2 (con operador de selección GATournament). No hay diferencias significativas (comportamiento similar) entre el tamaño de la población. Podríamos utilizar cualquiera de ellos, pero como la complejidad en espacio y en tiempo dependen del tamaño de la población, seleccionamos el valor **TP=100**.
- Entonces, nos queda analizar el comportamiento para los parámetros p_c y p_m en las columnas de la Tabla 2 referidas a TP=100.

Podríamos obtener otros comportamientos diferentes

Terminando de construir el sistema basado en un AG

- Operador de selección: "TOUR" (**GATournament**) y tamaño de población: **TP=100**.

Valores de p_c y p_m para tamaños de problemas en los que se obtiene solución óptima.

	p_m	nreinas
p_c=0.85	0.1, 0.075, 0.05 0.1, 0.075, 0.05, 0.025 0.075, 0.05, 0.025 0.05, 0.025 0.025	8 9 – 16, 20 9 – 16, 20 – 22 9 – 22, 30 9 – 40
p_c=0.9	0.1, 0.075, 0.05 0.1, 0.075, 0.05, 0.025 0.075, 0.05, 0.025 0.05, 0.025 0.025	11, 13 8 – 10, 12, 14 – 16, 19 8 – 10, 12, 14 – 16, 19, 21 8 – 10, 12, 14 – 21 8 – 10, 12, 14 – 40

Tanto para p_c=0.85 y p_c=0.9, el sistema tiene un comportamiento similar obteniendo valores 0 en prácticamente los mismos casos. Según la complejidad en tiempo, seleccionamos **p_c=0.85** por requerir menos recursos.

Podríamos obtener otros comportamientos diferentes

Terminando de construir el sistema basado en un AG

El sistema para **10000** generaciones, operador de cruce "**OnePointCrossover**", operador de mutación "**Flipmutator**" y utilizando los casos para ajuste de tamaños 8-40, tiene su mejor comportamiento (según los criterios del análisis) con:

- Operador de selección = TOUR (**GATournamentSelector**).
- Tamaño de la población de =**100**
- p_c = **0.85**

p_m = {0.1, 0.075, 0.05}	para 8 reinas
p_m = {0.1, 0.075, 0.05, 0.025}	para 9-16, 20 reinas
p_m = {0.075, 0.05, 0.025}	para 9-16, 20-22 reinas
p_m = {0.05, 0.025}	para 9-22, 30 reinas
p_m = {0.025}	para 9-40 reinas

Para este análisis/ajuste inicial, podemos concluir que los distintos parámetros están definidos excepto para "p_m" cuyo comportamiento nos induce que conforme crece el problema (mayor número de reinas) el valor de "p_m" debería decrecer.

En este análisis inicial podemos seleccionar (por optimalidad y requerir menos recursos) los valores de:

p_m=0.05 para nreinas≤8, y
p_m=0.025 para valores de 9≤nreinas≤40

Para terminar de ajustar este parámetro p_m debemos ejecutar nuevos casos con nuevos valores de p_m (disminuyendo su valor).

Terminando de construir el sistema basado en un AG

HACIA EL MANUAL DE ASIGNACIÓN

El manual de usuario nos indica cómo debemos ejecutar el software y qué tipo de información se nos pedirá asociada a los distintos parámetros. Lo que NO nos indica es qué valor concreto debemos asignar a cada parámetro en función del caso que se resolverá y para que tenga un comportamiento óptimo o aceptable. La función del "Manual de Asignación" debe ser esta.

El Manual de Asignación se debe obtener a partir de un estudio previo de casos y valores de parámetros. A partir del estudio realizado, el Manual de Asignación debe indicar claramente al usuario qué valores/métodos debe utilizar en los parámetros del sistema para un caso concreto a resolver.

Nota: Tanto un Manual de Usuario como un Manual de Asignación van dirigidos al usuario y por tanto, no deben incluir detalles técnicos.

A modo de ejemplo, un sencillo Manual de Asignación podría ser el siguiente:

Terminando de construir el sistema basado en un AG

Recomendaciones para la asignación de valores a parámetros para que el sistema tenga un comportamiento aceptable:

Al resolver un caso concreto del problema de las n-reinas le recomendamos los siguientes valores para los parámetros que le pedirá el sistema con el fin de obtener buenos resultados. Algunos valores recomendados van en función del valor de "n" (nº de reinas: tamaño del caso) correspondiente al parámetro 1º:

Valor para el parámetro 2º: 100 (valor recomendado)

Valor para el parámetro 3º: 10000 (valor recomendado)

Valor para el parámetro 4º: TOUR (valor recomendado)

Valor para el parámetro 5º: ONEP (valor recomendado)

Valor para el parámetro 6º: FLIP (valor recomendado)

Valor para el parámetro 7º: 0.85 (valor recomendado)

Valor para el parámetro 8º:
- Si --- ≤ n ≤ ---: xx (valor recomendado)
- Si --- ≤ n ≤ ---: yy (valor recomendado)
- .....
- Si --- ≤ n ≤ ---, zz (valor recomendado)

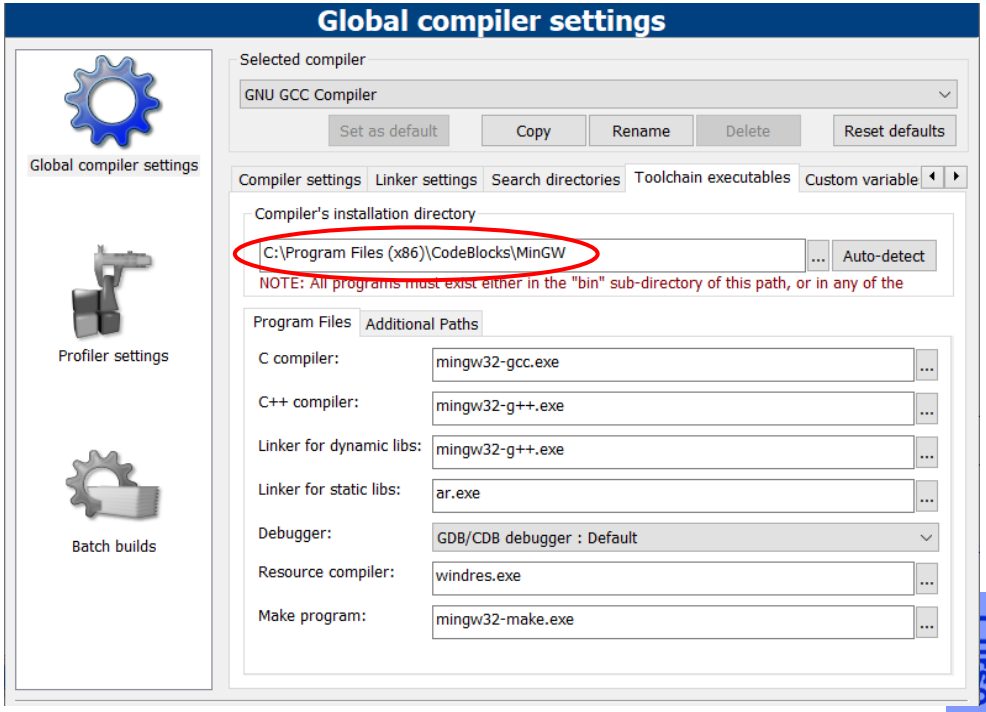
Sobre la compilación con GAlib

Su uso con Code::Blocks para Windows

En las transparencias anteriores se indica cómo se utiliza la librería GAlib en un proyecto desarrollado en Code::Blocks para Windows. Para que no haya ningún problema en el uso de la librería se debe tener en cuenta lo siguiente:

- Primero, debe descargarse la versión de Code::Blocks con el compilador "MinGW" incluido (codeblocks-17.12mingw-setup.exe) e instalarlo (disponible de manera pública en https://sourceforge.net/projects/codeblocks/files/Binaries/17.12/Windows/)

- Segundo, hay que asegurarse que no tenemos ningún otro compilador C/C++ en el ordenador, o que en la compilación de nuestro proyecto se esté utilizando el compilador MinGW instalado con Code::Blocks (lo podemos comprobar dentro de Code::Blocks en setting-compiler-Toolchain Executables) (ver figura).



Sobre la compilación con GAlib

Su uso con Code::Blocks para Windows

En caso de no querer realizar lo anteriormente expuesto, se recomienda la opción de utilizar el entorno virtual EVA ("INFORMATICA_WINDOWS").

En el entorno virtual se puede utilizar Code::Blocks, teniendo en cuenta que se utilizará únicamente para compilar y crear el .exe de nuestro proyecto.

Con dicho .exe podremos ya, en nuestro ordenador, realizar las pruebas indicadas en el guion de la práctica.